

Reinforcement Learning

Policy Iteration & Value Iteration

Min-hwan Oh

Seoul National University

Policy Evaluation & Improvement

- Policy evaluation
 - ▶ Given policy π , compute value function v_π
 - ▶ v_π exists and is unique if $\gamma < 1$ or termination is guaranteed for all states under policy π .
 - ▶ Policy evaluation converges for finite MDPs.

Policy Evaluation & Improvement

- Policy evaluation

- ▶ Given policy π , compute value function v_π
- ▶ v_π exists and is unique if $\gamma < 1$ or termination is guaranteed for all states under policy π .
- ▶ Policy evaluation converges for finite MDPs.

- Policy improvement

- ▶ Given value function v_π , compute improved policy π'
- ▶ Policy improvement theorem:

$$\forall s \in \mathcal{S}, q_\pi(s, \pi'(s)) \geq v_\pi(s) \Rightarrow \forall s \in \mathcal{S}, v_{\pi'}(s) \geq v_\pi(s)$$

- ▶ Polynomial time convergence (in number of n states and k actions) even though k^n policies.

But, what is our goal?

But, what is our goal?
We want to find an optimal policy!

Policy Iteration

- Iteration of policy evaluation and improvement

$$\pi_0 \xrightarrow{E} v_{\pi_0} \xrightarrow{I} \pi_1 \xrightarrow{E} v_{\pi_1} \xrightarrow{I} \pi_2 \xrightarrow{E} \cdots \xrightarrow{I} \pi_* \xrightarrow{E} v_{\pi_*}$$

- policy evaluation E , policy improvement I

Policy Iteration

- Iteration of policy evaluation and improvement

$$\pi_0 \xrightarrow{E} v_{\pi_0} \xrightarrow{I} \pi_1 \xrightarrow{E} v_{\pi_1} \xrightarrow{I} \pi_2 \xrightarrow{E} \cdots \xrightarrow{I} \pi_* \xrightarrow{E} v_{\pi_*}$$

- policy evaluation E , policy improvement I
- Each policy is guaranteed to be a strict improvement over the previous policy, except when?

Policy Iteration

- Iteration of policy evaluation and improvement

$$\pi_0 \xrightarrow{E} v_{\pi_0} \xrightarrow{I} \pi_1 \xrightarrow{E} v_{\pi_1} \xrightarrow{I} \pi_2 \xrightarrow{E} \cdots \xrightarrow{I} \pi_* \xrightarrow{E} v_{\pi_*}$$

- policy evaluation E , policy improvement I
- Each policy is guaranteed to be a strict improvement over the previous policy, except when? When the policy is already optimal!

Policy Iteration Pseudocode

Policy Iteration (using iterative policy evaluation) for estimating $\pi \approx \pi_*$

1. Initialization

$V(s) \in \mathbb{R}$ and $\pi(s) \in \mathcal{A}(s)$ arbitrarily for all $s \in \mathcal{S}$

2. Policy Evaluation

Loop:

$\Delta \leftarrow 0$

Loop for each $s \in \mathcal{S}$:

$v \leftarrow V(s)$

$V(s) \leftarrow \sum_{s',r} p(s',r|s,\pi(s)) [r + \gamma V(s')]$

$\Delta \leftarrow \max(\Delta, |v - V(s)|)$

until $\Delta < \theta$ (a small positive number determining the accuracy of estimation)

3. Policy Improvement

policy-stable $\leftarrow$ *true*

For each $s \in \mathcal{S}$:

old-action $\leftarrow \pi(s)$

$\pi(s) \leftarrow \arg\max_a \sum_{s',r} p(s',r|s,a) [r + \gamma V(s')]$

If *old-action* $\neq \pi(s)$, then *policy-stable* $\leftarrow$ *false*

If *policy-stable*, then stop and return $V \approx v_*$ and $\pi \approx \pi_*$; else go to 2

Policy Iteration in Gridworld

Compute v_π for random policy π

$k = 0$

0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0

$k = 3$

0.0	-2.4	-2.9	-3.0
-2.4	-2.9	-3.0	-2.9
-2.9	-3.0	-2.9	-2.4
-3.0	-2.9	-2.4	0.0

π : initial policy

	↔	↔	↔
↔	↔	↔	↔
↔	↔	↔	↔
↔	↔	↔	

$k = 1$

0.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	0.0

$k = 10$

0.0	-6.1	-8.4	-9.0
-6.1	-7.7	-8.4	-8.4
-8.4	-8.4	-7.7	-6.1
-9.0	-8.4	-6.1	0.0

$k = 2$

0.0	-1.7	-2.0	-2.0
-1.7	-2.0	-2.0	-2.0
-2.0	-2.0	-2.0	-1.7
-2.0	-2.0	-1.7	0.0

$k = 1000$

0.0	-14	-20	-22
-14	-18	-20	-20
-20	-20	-18	-14
-22	-20	-14	0.0

Policy Iteration in Gridworld

Compute v_π for random policy π

$k = 0$

0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0

$k = 3$

0.0	-2.4	-2.9	-3.0
-2.4	-2.9	-3.0	-2.9
-2.9	-3.0	-2.9	-2.4
-3.0	-2.9	-2.4	0.0

$k = 1$

0.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	0.0

$k = 10$

0.0	-6.1	-8.4	-9.0
-6.1	-7.7	-8.4	-8.4
-8.4	-8.4	-7.7	-6.1
-9.0	-8.4	-6.1	0.0

$k = 2$

0.0	-1.7	-2.0	-2.0
-1.7	-2.0	-2.0	-2.0
-2.0	-2.0	-2.0	-1.7
-2.0	-2.0	-1.7	0.0

$k = 1000$

0.0	-14	-20	-22
-14	-18	-20	-20
-20	-20	-18	-14
-22	-20	-14	0.0

π : initial policy

	↕	↕	↕
↕	↕	↕	↕
↕	↕	↕	↕
↕	↕	↕	

π' : new policy

	←	←	↖
↑	↖	↖	↓
↑	↗	↗	↓
↗	→	→	

Value Iteration

- Drawback of policy iteration: each iteration involves policy evaluation, requiring multiple sweeps through the state set
- Value iteration: evaluate policy for one sweep!
 - ▶ does not evaluate v_π fully for given π

Value Iteration Pseudocode

Value Iteration, for estimating $\pi \approx \pi_*$

Algorithm parameter: a small threshold $\theta > 0$ determining accuracy of estimation
Initialize $V(s)$, for all $s \in \mathcal{S}^+$, arbitrarily except that $V(\text{terminal}) = 0$

Loop:

```
|  $\Delta \leftarrow 0$   
|   Loop for each  $s \in \mathcal{S}$ :  
|      $v \leftarrow V(s)$   
|      $V(s) \leftarrow \max_a \sum_{s',r} p(s',r|s,a) [r + \gamma V(s')]$   
|      $\Delta \leftarrow \max(\Delta, |v - V(s)|)$   
until  $\Delta < \theta$ 
```

Output a deterministic policy, $\pi \approx \pi_*$, such that

$$\pi(s) = \arg \max_a \sum_{s',r} p(s',r|s,a) [r + \gamma V(s')]$$

Value Iteration vs Policy Iteration

Value Iteration, for estimating $\pi \approx \pi_*$

Algorithm parameter: a small threshold $\theta > 0$ determining accuracy of estimation
Initialize $V(s)$, for all $s \in \mathcal{S}^+$, arbitrarily except that $V(\text{terminal}) = 0$

Loop:

```
|  $\Delta \leftarrow 0$   
| Loop for each  $s \in \mathcal{S}$ :  
|    $v \leftarrow V(s)$   
|    $V(s) \leftarrow \max_a \sum_{s',r} p(s',r|s,a)[r + \gamma V(s')]$   
|    $\Delta \leftarrow \max(\Delta, |v - V(s)|)$   
until  $\Delta < \theta$ 
```

Output a deterministic policy, $\pi \approx \pi_*$, such that
$$\pi(s) = \arg \max_a \sum_{s',r} p(s',r|s,a)[r + \gamma V(s')]$$

Policy Iteration (using iterative policy evaluation) for estimating $\pi \approx \pi_*$

1. Initialization

$V(s) \in \mathbb{R}$ and $\pi(s) \in \mathcal{A}(s)$ arbitrarily for all $s \in \mathcal{S}$

2. Policy Evaluation

Loop:

```
 $\Delta \leftarrow 0$   
Loop for each  $s \in \mathcal{S}$ :  
   $v \leftarrow V(s)$   
   $V(s) \leftarrow \sum_{s',r} p(s',r|s,\pi(s))[r + \gamma V(s')]$   
   $\Delta \leftarrow \max(\Delta, |v - V(s)|)$ 
```

until $\Delta < \theta$ (a small positive number determining the accuracy of estimation)

3. Policy Improvement

policy-stable $\leftarrow$ true

For each $s \in \mathcal{S}$:

```
  old-action  $\leftarrow \pi(s)$   
   $\pi(s) \leftarrow \arg \max_a \sum_{s',r} p(s',r|s,a)[r + \gamma V(s')]$   
  If old-action  $\neq \pi(s)$ , then policy-stable  $\leftarrow$  false
```

If *policy-stable*, then stop and return $V \approx v_*$ and $\pi \approx \pi_*$; else go to 2

- What is the difference between value iteration and policy iteration?

Generalized Policy Iteration

- Policy iteration consists of two simultaneous, interacting processes
 - ▶ policy evaluation: making the value function consistent with the current policy
 - ▶ policy improvement: making the policy greedy with respect to the current value function
- As long as both processes continue to update all states, we have convergence to the optimal value function and an optimal policy
- Generalized policy iteration (GPI): general framework of letting policy-evaluation and policy-improvement processes interact

Illustration of Policy Iteration

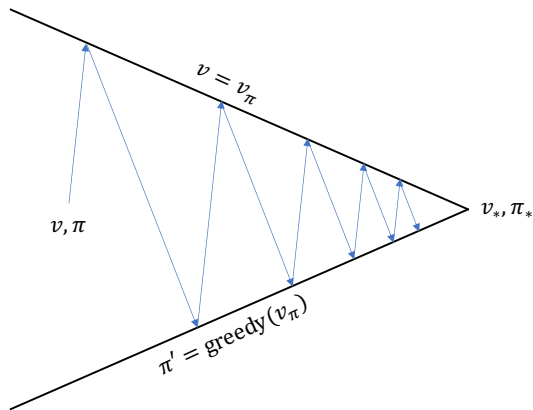
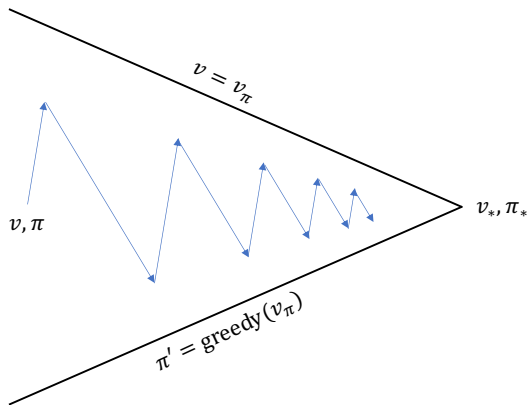


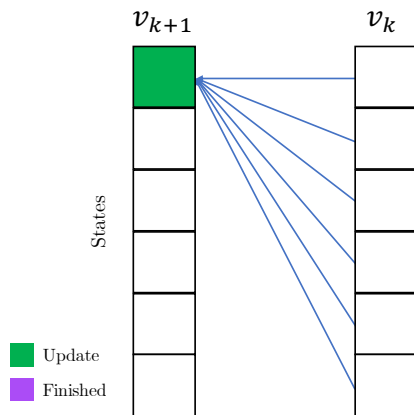
Illustration of Generalized Policy Iteration (GPI)



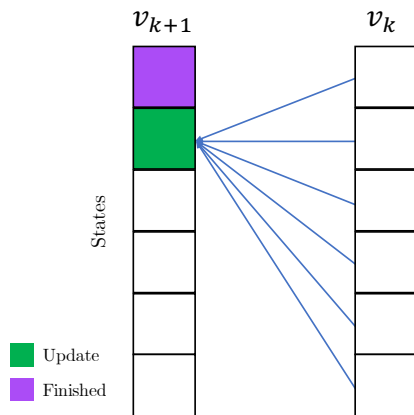
Notes on GPI

- For almost all RL methods (including DP methods), we can view them as GPI
- GPI is the general idea of two interacting processes: an approximate policy and an approximate value function

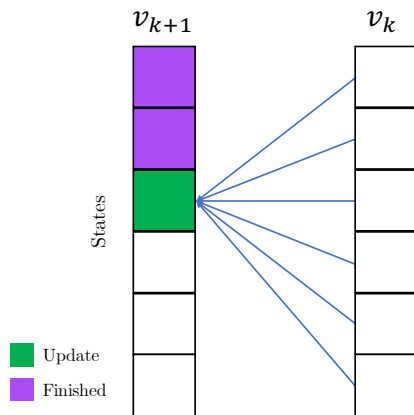
Synchronous Updates with Two Arrays



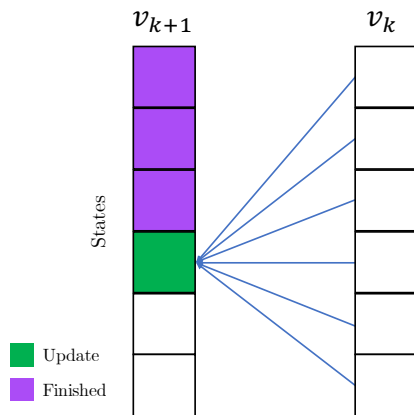
Synchronous Updates with Two Arrays



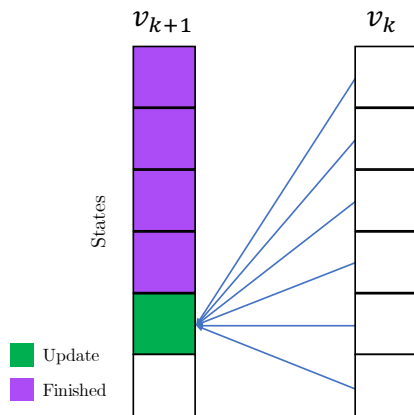
Synchronous Updates with Two Arrays



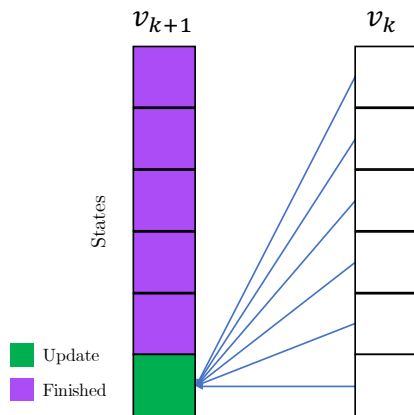
Synchronous Updates with Two Arrays



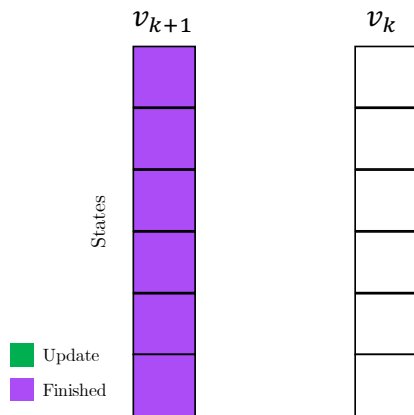
Synchronous Updates with Two Arrays



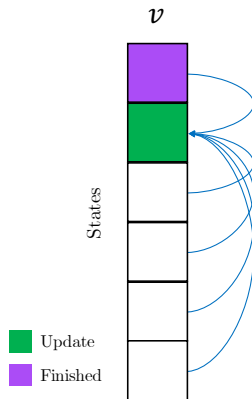
Synchronous Updates with Two Arrays



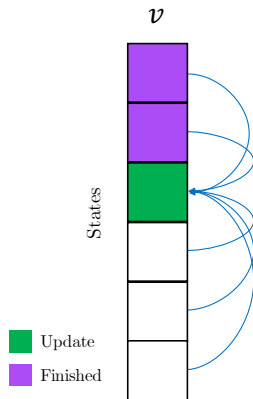
Synchronous Updates with Two Arrays



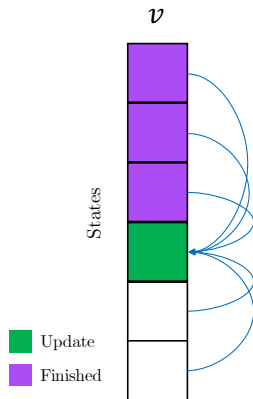
Synchronous “In-place” Updates



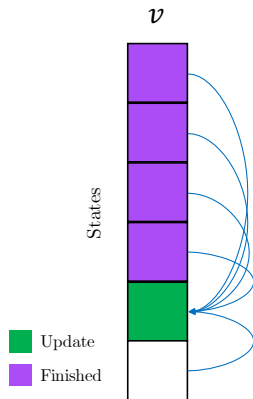
Synchronous “In-place” Updates



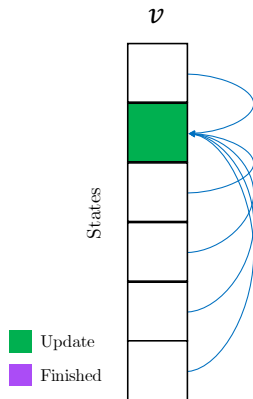
Synchronous “In-place” Updates



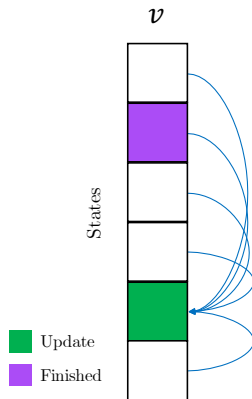
Synchronous “In-place” Updates



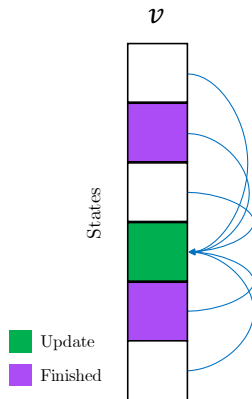
Asynchronous “In-place” Updates



Asynchronous “In-place” Updates



Asynchronous “In-place” Updates



Efficiency of Dynamic Programming

- DP may not be practical for very large problems
- But, compared with other methods for solving MDPs, DP methods are actually quite efficient
 - ▶ Finds an optimal policy in polynomial times of $|S|$ and $|\mathcal{A}|$
 - ▶ Note that total number of (deterministic) policies is $|\mathcal{A}|^{|S|}$
 - ▶ Hence, DP is exponentially faster than any direct search in policy space

Efficiency of Dynamic Programming

- DP may not be practical for very large problems
- But, compared with other methods for solving MDPs, DP methods are actually quite efficient
 - ▶ Finds an optimal policy in polynomial times of $|S|$ and $|A|$
 - ▶ Note that total number of (deterministic) policies is $|A|^{|S|}$
 - ▶ Hence, DP is exponentially faster than any direct search in policy space
- DP is often thought to be of limited applicability because of the *curse of dimensionality*
 - ▶ The number of states $|S|$ often grows exponentially
 - ▶ Large state sets do create difficulties, but these are inherent difficulties of the problem, not of DP as a solution method

Efficiency of Dynamic Programming

- DP may not be practical for very large problems
- But, compared with other methods for solving MDPs, DP methods are actually quite efficient
 - ▶ Finds an optimal policy in polynomial times of $|S|$ and $|\mathcal{A}|$
 - ▶ Note that total number of (deterministic) policies is $|\mathcal{A}|^{|S|}$
 - ▶ Hence, DP is exponentially faster than any direct search in policy space
- DP is often thought to be of limited applicability because of the *curse of dimensionality*
 - ▶ The number of states $|S|$ often grows exponentially
 - ▶ Large state sets do create difficulties, but these are inherent difficulties of the problem, not of DP as a solution method
- DP can solve MDPs with millions of states with today's computers
 - ▶ Usually converge much faster than their theoretical worst-case run times
 - ▶ With large state spaces, asynchronous DP methods are often preferred

Next class: Monte Carlo Methods (Chapter 5 of Sutton & Barto)